

# Data Structures and Algorithms Analysis CoSc3111\_

June 15, 2025

Compiled by: Natan Asrat

[https://www.linkedin.com/in/natan\\_asrat](https://www.linkedin.com/in/natan_asrat)

# Data Structures and Algorithms Analysis

- Computer program is a solution to a problem which contains two basic entities:
  - The way data are organized in a computer's memory which referred as Data Structure and
  - The sequence of computational steps to solve a problem is said to be an algorithm
  - "program = algorithm + data structure"

# Data Structures and Algorithms Analysis

- Classification of data structure:
  - Primitive: closest to the machine level instruction

# Data Structures and Algorithms Analysis

- Classification of data structure:
  - Non-Primitive: data item are not operated closest to the machine level instruction.
    - Linear: data items stored in sequence
      - Example: Array, stack and Queue,
      - Operations:
        - Traversal - Processing each element in the list
        - Search - Finding the location of the element with a given value.
        - Insertion - Adding a new element to the list.
        - Deletion - Removing an element from the list.
        - Sorting - Arranging the elements in some type of order.
        - Merging - Combining two lists into a single list.
    - Non-Linear: in which there is order of data items
      - Example: Trees and Graphs



# Data Structures and Algorithms Analysis

- Algorithm: finite set of instructions which, if followed, accomplish a particular task.
  - Criteria
    - Input: there are zero or more quantities which are externally supplied;
    - Output: at least one quantity is produced;
    - Definiteness: each instruction must be clear and unambiguous;
    - Finiteness: for all cases the algorithm will terminate after a finite number of steps;

# Data Structures and Algorithms Analysis

- Algorithm:
  - Algorithm analysis: determining the amount of computing time and storage space required by different algorithms.
    - produce a function  $T(n)$  that describes the algorithm in terms of the operations performed in order to measure the complexity of the algorithm.
  - Complexity Analysis: is systematic study of the cost of computation,
    - measured either in time units or in operations performed, or in the amount of storage space required.

# Data Structures and Algorithms Analysis

- Algorithm:
  - Asymptotic analysis: how the running time of an algorithm increases with the size of the input in the limit, as the size of the input increases without bound.
    - Notations:
      - Big-Oh Notation: an upper bound on the growth rate of a function,
      - Big-Omega Notation: lower bound;
        - $f(n) = \text{Big-omega}(g(n))$  - means that the growth rate of  $f(n)$  is greater than or equal to  $g(n)$ .
      - Theta Notation: have the same rate of growth (tight bound)
      - Little-o Notation:  $f(n) = o(g(n))$  means  $f(n)$  has less growth rate compared to  $g(n)$ .
      - Little-Omega Notation: a lower bound that is not asymptotically tight.

# Data Structures and Algorithms Analysis

- Searching: looking for a specific element in a list of items or determining that the item is not in the list.
  - Linear/Sequential searching
    - Works with unsorted List
    - Loop through the list starting at the first element
    - time complexity  $O(n)$ .
  - Binary Searching
    - Works on ordered/sorted list
    - divide and conquer method:
      - locates mid-point and identify which have the target belongs to since the list is sorted.
      - It performs this recursively till the item found or the list is empty(target is not in the list)
    - time complexity -  $O(\log_2 n)$ .



# Data Structures and Algorithms Analysis

- Sorting: reordering a list of items in either increasing or decreasing order.
  - Insertion Sort: most instinctive type of sorting
    - finds the location for an element and move all others up
    - starts with the left most two items and put them in order
    - add 3rd item to the relatively sorted portion and find its correct location
    - Comparisons  $\Rightarrow O(n^2)$ ; Swaps  $\Rightarrow O(n^2)$

# Data Structures and Algorithms Analysis

- Sorting:
  - Selection Sort: loop through and select smallest, put on left side, repeat
    - Comparisons  $\Rightarrow O(n^2)$ ; Swaps  $\Rightarrow O(n)$
  - Bubble Sort: simplest to implement but slowest on very large inputs.
    - loop through and swap adjacent elements if out of order
    - Comparisons  $\Rightarrow O(n^2)$ ; Swaps  $\Rightarrow O(n^2)$

# Data Structures and Algorithms Analysis

- Struct: object consisting of multiple data items that may be of different types.
  - Single Linked lists: self-referential structure, a chain of structs/Nodes
    - Each node in the list contains data items and pointer to next node

# Data Structures and Algorithms Analysis

- Struct - Single Linked lists:
  - Operations:
    - Creating Linked Lists
    - Adding node to the list

```
if (start == NULL) // If list is empty, then new node become start node  
    start = temp;  
else {  
    temp2 = start; // list not empty! Thus temp2 starts from first node  
    while (temp2->next != NULL) { // till the current last node found navigates forward  
        temp2 = temp2->next; // Move to next link in chain  
    }  
    temp2->next = temp; // new node (temp) appended at the end of the list  
}
```



# Data Structures and Algorithms Analysis

- Struct - Single Linked lists:

- Operations:

- Displaying the list of Nodes
    - Deleting a node from the list

1. Look at the start pointer. If it is NULL, then the list is empty, so print out a "No nodes to delete" message.
2. Make temp1 point to whatever the start pointer is pointing to.
3. If the next pointer of what temp1 indicates is NULL, then we've found the last node of the list, so jump to step 7.
4. Make another pointer, temp2, point to the current node in the list.
5. Make temp1 point to the next item in the list.
6. Go to step 3.
7. If you get this far, then the temporary pointer, temp1, should point to the last item in the list and the other temporary pointer, temp2, should point to the last-but-one item.
8. Delete the node pointed to by temp1.
9. Mark the next pointer of the node pointed to by temp2 as NULL - it is the new last node.

# Data Structures and Algorithms Analysis

- Struct: object consisting of multiple data items that may be of different types.
  - Stack: ordered list in which all insertions and deletions are made at one end, called the "top"
    - Operations:
      - Push: To add an element to the stack□
      - Pop: To remove an element from the stock□
      - Peek: To look at elements in the stack without removing them

# Data Structures and Algorithms Analysis

- Struct - Stack:
  - Implementations:
    - Array implementation of stack

## **Push(n): Adding item at top**

*top is the current top of stack and n is its maximum size*

Begin

if (top == n) then stack is full;

else

top := top+1;

stack[top] := item;

end;

## **Pop(): removing item from stack top**

*remove top element from the stack and put it in the item*

Begin:

if (top == 0) then stack is empty;

else

item := stack[top];

top := top-1;

end;

# Data Structures and Algorithms Analysis

- Struct - Stack:
  - Implementations:
    - Linked list implementation:
      - stack is adding items at the end of the chain (Push item)
      - deleting the end node of the list (Pop item)



# Data Structures and Algorithms Analysis

- Struct - Stack:
  - Applications:
    - to evaluate arithmetic expressions. (Postfix Expressions)
    - Infix to Postfix Transformation
    - execution of recursive programs
    - designing the compiler in operating system
    - function calls including recursion.
    - reversing a List
    - convert Decimal to Binary
    - backtracking

# Data Structures and Algorithms Analysis

- Struct:
  - Queue: ordered list in which all insertions at one end called REAR and deletions are made at another end called FRONT
    - Operations
      - Enqueue - Inserts an item at the rear
        - overflow error occurs if the queue is full.
      - Dequeue - Removes an item from the front and returns it to the user.
        - underflow error occurs if the queue is empty.

# Data Structures and Algorithms Analysis

- Struct - Queue:
  - Implementations:
    - Simple Array implementation (static memory)
      - even though there are free released spaces at the front of the array the queue implementation might generate overflow message.
    - Circular array implementation
      - freed spaces are re-used to store data, copy data to left
    - Linked list implementation

# Data Structures and Algorithms Analysis

- Struct - Queue:
  - Deque (pronounced as Deck): Double Ended Queue in which insertion and deletion can occur at either end.
  - Priority Queue:- each data has an associated key provided at insertion time.



# Data Structures and Algorithms Analysis

- Struct - Queue:
  - Applications:
    - Print server
    - Disk Driver- queue of disk input/output requests
    - Task scheduler in multiprocessing system- priority queues of processes
    - Telephone calls in a busy environment
    - Simulation of waiting line- e.g. supermarkets checkout, customer service line...

# Data Structures and Algorithms Analysis

- Struct:
  - Tree: set of nodes and edges that connect pairs of nodes that connect pairs of nodes.
    - abstract model of a hierarchical structure.

# Data Structures and Algorithms Analysis

- Struct – Tree

- Terminologies:

- Root: a node without a parent.
    - Internal node: a node with at least one child.
    - External (leaf) node: a node without a child.
    - Ancestors of a node: parent, grandparent, grand-grandparent, etc of a node.
    - Descendants of a node: children, grandchildren, grand-grandchildren etc of a node.
    - Depth of a node: number of ancestors or length of the path from the root to the node.
    - Height of a tree: depth of the deepest node.
    - Subtree: a tree consisting of a node and its descendants.

# Data Structures and Algorithms Analysis

- Struct – Tree

- Terminologies:

- Binary tree: a tree in which each node has at most two children called left child and right child.
    - Binary search tree (ordered binary tree): a binary tree that may be empty, but if it is not empty it satisfies the following.
      - Every node has a key and no two elements have the same key.
      - The keys in the right subtree are larger than the keys in the root.
      - The keys in the left subtree are smaller than the keys in the root.
      - The left and the right subtrees are also binary search trees.



# Data Structures and Algorithms Analysis

- Struct – Tree

- Operations

- Tree structure definition

- Insertion

- Tree Traversing

- Pre order traversal - traversing binary tree in the order of parent, left and right.

- Inorder traversal - traversing binary tree in the order of left, parent and right.

- Postorder traversal - traversing binary tree in the order of left, right and parent.

- Searching

# Data Structures and Algorithms Analysis

- Struct – Tree

- Operations – Deletion

- Cases:

- Case 1: Deleting a leaf node, straightforward operation.
        - delete the leaf node and make the pointer of the deleted node parent NULL (right or left child pointer)
      - Case 2: Deleting a node having only one child
      - Case 3: Deleting a node having two children
      - Case 4: Deleting the root node

# Data Structures and Algorithms Analysis

- Struct - Tree

- Operations - Deletion

- Approaches for Cases 2, 3, 4 - Deletion by copying:

- Copy the node containing the largest element in the left (or the smallest element in the right) to the node containing the element to be deleted
      - Delete the Copied Node

# Data Structures and Algorithms Analysis

- Struct - Tree
  - Operations - Deletion
    - Approaches for Cases 2, 3, 4 - Deletion by merging
      - Case 2 (1 child):
        - Promote a child (left or right) of deleted node as a child (left or right) for the parent of the deleted node keeping the definition of BST.



# Data Structures and Algorithms Analysis

- Struct - Tree
  - Operations - Deletion
    - Approaches for Cases 2, 3, 4 - Deletion by merging
      - Case 3 (2 children):
        - Promote a child (left or right) of deleted node as a child for the parent of the deleted node.
        - The other child of the deleted node is made the left child of the node containing smallest element in the right of the deleted node or the right child of the deleted node is made the right child of the node containing largest element in the left of the deleted node

# Data Structures and Algorithms Analysis

- Struct - Tree

- Operations - Deletion

- Approaches for Cases 2, 3, 4 - Deletion by merging

- Case 4 (root):

- the root node pointer is made to point to the right child or left child
        - the left child/right child of the root node is made the left child/right child of the node containing the smallest element/largest in the right/left of the root node respectively.

# Data Structures and Algorithms Analysis

- **Advanced Sorting and Searching Algorithms**

- Shell Sort: improvement of insertion sort, creates a reasonable order first.
  - time complexity is  $O(n^{3/2})$
  - Algorithm:
    - Choose gap  $g_k$  between elements to be partly ordered.
    - Generate a sequence (called increment sequence)  $g_k, g_{k-1}, \dots, g_2, g_1$  where for each sequence  $g_i$ ,  $A[j] \leq A[j+g_i]$  for  $0 \leq j \leq n-1-g_i$  and  $k \geq i \geq 1$

# Data Structures and Algorithms Analysis

- **Advanced Sorting and Searching Algorithms**

- Quick sort: fastest known algorithm, divide and conquer strategy
  - worst case its complexity is  $O(n^2)$
  - expected complexity is  $O(n \log n)$ .
  - Algorithm
    - Choose a pivot value (mostly the first element is taken as the pivot value)
    - Position the pivot element and partition the list so that:
      - the left part has items less than or equal to the pivot value
      - the right part has items greater than or equal to the pivot value
    - Recursively sort the left part
    - Recursively sort the right part



# Data Structures and Algorithms Analysis

- **Advanced Sorting and Searching Algorithms**

- Heap Sort: first convert the list in to a heap tree.
  - Heap tree is a binary tree in which each node has a value greater than both its children (if any).
  - process called "adjust" to accomplish its task (building a heap tree) whenever a value is larger than its parent.
  - time complexity -  $O(n \log n)$ .

# Data Structures and Algorithms Analysis

- **Advanced Sorting and Searching Algorithms**

- Heap Sort - Algorithm:

- Construct a binary tree

- The root node corresponds to `Data[0]`.

- If we consider the index associated with a particular node to be  $i$ , then the left child of this node corresponds to the element with index  $2i+1$  and the right child corresponds to the element with index  $2i+2$ . If any or both of these elements do not exist in the array, then the corresponding child node does not exist either.

- Construct the heap tree from initial binary tree using "adjust" process. Sort by swapping the root value with the lowest, right most value and deleting the lowest, right most value and inserting the deleted value in the array in its proper position.

# Data Structures and Algorithms Analysis

- **Advanced Sorting and Searching Algorithms**

- Merge sort: divide and conquer strategy
  - time complexity is  $O(n \log n)$ .
  - Algorithm:
    - Divide the array in to two halves.
    - Recursively sort the first  $n/2$  items.
    - Recursively sort the last  $n/2$  items.
    - Merge sorted items (using an auxiliary array).